FAST National University of Computing

and Emerging Sciences

Assignment 2

Input Specific Neural Networks

Implementation, Training, and Evaluation

Submitted by:

Ayan Bin Moeen

Student ID: 23I-0124

Course: Artificial Neural Networks

Instructor: Saif Ur Rehman

Date: April 2026

1. Abstract

This report presents the implementation and evaluation of Input Specific Neural Networks (ISNNs), a novel neural network architecture proposed by Jadoon et al. (2025). ISNNs extend traditional constrained neural networks by simultaneously enforcing multiple structural constraints on different input variables — specifically convexity, monotonicity combined with convexity, simple monotonicity, and arbitrary relationships. Two architectures, ISNN-1 and ISNN-2, are implemented using two different approaches: a PyTorch-based implementation that leverages automatic differentiation, and a fully manual NumPy implementation where backpropagation is computed by hand following the chain-rule algorithm. Both implementations are trained and evaluated on two toy datasets described in the paper, and the resulting loss curves and behavioral plots are produced and discussed.

2. Introduction

Neural networks have become powerful function approximators across a wide variety of domains. However, a common limitation is their black-box nature — they do not inherently encode or enforce known structural relationships between inputs and outputs. In scientific machine learning, such structural relationships are frequently known in advance from physical laws or mathematical principles, and failing to encode them can lead to poor generalisation, particularly outside the training domain.

Prior work has addressed this by constructing architectures that enforce a single type of constraint — for example, Input Convex Neural Networks (ICNNs) enforce convexity with respect to certain inputs, while monotone networks enforce monotonicity. However, most existing approaches impose only one type of constraint on a set of inputs, leaving the others entirely unconstrained.

The paper "Input Specific Neural Networks" by Jadoon et al. (2025) proposes a new architecture, the ISNN, that addresses this gap by simultaneously enforcing multiple, different constraints on different sets of inputs for a scalar-valued output. Specifically, an ISNN can enforce:

- Convexity with respect to inputs x0
- Convexity AND monotonically non-decreasing behaviour with respect to inputs y0
- Only monotonically non-decreasing behaviour with respect to inputs t0
- Arbitrary (unconstrained) relationships with respect to inputs z0

Two architectures are proposed — ISNN-1 and ISNN-2 — which differ in how the input branches interact. This report implements both architectures from scratch using PyTorch and NumPy, trains them on two toy problems from the paper, and evaluates their performance against an unconstrained Feed-Forward Neural Network (FFNN) baseline.

3. Dataset Description

Two toy datasets are generated following the exact specifications of Section 3.1 of the paper. Latin Hypercube Sampling (LHS) is used for all data generation, as specified in the paper, to ensure uniform space-filling coverage of the input domain.

3.1 Toy Problem 1 — Additively Split Function

The first toy problem uses the following additively split function from Equation 12 of the paper:

$$f = \exp(-0.5x) + \log(1 + \exp(0.4y)) + \tanh(t) + \sin(z) - 0.4$$

This function is convex in x, convex and monotonically increasing in y, monotonically increasing in t, and arbitrary in z — matching exactly the constraint categories that ISNN enforces.

Property	Training Set	Test Set
Number of samples	500	5000
Input domain	[0.0, 4.0]	[0.0, 6.0]
Sampling method	Latin Hypercube Sampling	Latin Hypercube Sampling
Number of inputs	4 (x, y, t, z)	4 (x, y, t, z)
Output	Scalar f	Scalar f

3.2 Toy Problem 2 — Multiplicatively Split Function

The second toy problem uses the following multiplicatively split function from Equations 13-14 of the paper:

$$g(x, y, z, t) = f_x \cdot f_y \cdot f_z \cdot f_t$$

$$\text{where } f_x = \exp(-0.3x),\ \ f_y = (0.15y)^2,\ \ f_t = \tanh(0.3t),\ \ f_z = 0.2\sin(0.5z + 2) + 0.5$$

This function shares the same structural properties as Toy Problem 1 — convex in x, convex and monotonically increasing in y, monotonically increasing in t, and arbitrary in z — but with a more complex multiplicative interaction that makes extrapolation significantly harder.

Property	Training Set	Test Set
Number of samples	500	5000
Input domain	[0.0, 4.0]	[0.0, 10.0]
Sampling method	Latin Hypercube Sampling	Latin Hypercube Sampling
Number of inputs	4 (x, y, t, z)	4 (x, y, t, z)
Output	Scalar g	Scalar g

Note that the test domain for Toy Problem 2 extends to [0, 10] compared to the training domain of [0, 4], creating a significantly larger extrapolation region. This is intentional — it stress-tests the ability of constrained architectures to generalise beyond training data.

4. Neural Network Architectures

4.1 Activation Functions

As specified in Equation 11 of the paper, the following activation functions are used:

- $\sigma_{mc}(x) = \log(1 + \exp(x))$ — Softplus: convex and monotonically non-decreasing. Used for branches requiring convexity.
- $\sigma_m(x) = \sigma_a(x) = 1/(1 + \exp(-x))$ — Sigmoid: monotonically non-decreasing. Used for monotone-only and arbitrary branches.

In the implementation, PyTorch's torch.nn.functional.softplus is used instead of the naive log(1 + exp(x)) formula to ensure numerical stability and prevent overflow for large input values.

4.2 Constrained Weight Parameterisation

Several weight matrices in both architectures must be non-negative to enforce the structural constraints. Rather than projecting weights after each update (which can be unstable), a reparameterisation approach is used: raw unconstrained parameters W_raw are stored and the effective weight is computed as:

$$\mathbf{W_effective = softplus(W_raw) = log(1 + exp(W_raw)) \geq 0}$$

This ensures weights are always non-negative while allowing unconstrained optimisation through the raw parameters. The gradient flows through the softplus transformation via the chain rule automatically in PyTorch, and is computed manually in the NumPy implementation.

4.3 ISNN-1 Architecture

ISNN-1 (Figure 1 in the paper) processes inputs through four separate trunk networks before combining them. The architecture follows Equations 1 through 5 of the paper:

- y-branch (Eq. 1): σ_{mc} activation, non-negative weights $W[yy]_h$. Processes y0 through Hy layers.
- z-branch (Eq. 2): σ_a activation, unconstrained weights $W[zz]_h$. Processes z0 through Hz layers.
- t-branch (Eq. 3): σ_m activation, non-negative weights $W[tt]_h$. Processes t0 through Ht layers.
- x-branch first layer F0 (Eq. 4): Combines x0, yH_y, zH_z, tH_t. Weight on x0 is unconstrained; weights from y and t branches are non-negative.
- x-branch subsequent layers Fh (Eq. 5): σ_{mc} activation, non-negative weights $W[xx]_h$ for $h \geq 1$.

The key characteristic of ISNN-1 is that the other branches (y, z, t) only feed into the x-branch at the first layer F0. Subsequent x-layers are purely recurrent within the x-branch.

For this implementation, the paper's configuration of 2 hidden layers with 10 neurons each for each input branch is used, giving approximately 1600 trainable parameters.

4.4 ISNN-2 Architecture

ISNN-2 (Figure 2 in the paper) is a more interconnected variant described by Equations 6 through 10. The key difference from ISNN-1 is that all branches have the same depth H, and the x-branch receives skip connections from x0, y_h, z_h, and t_h at every layer, not just the first.

- y-branch (Eq. 6): σ_mc, non-negative W[yy]_h, h = 0...H-2
- z-branch (Eq. 7): σ_a, unconstrained W[zz]_h, h = 0...H-2
- t-branch (Eq. 8): σ_m, non-negative W[tt]_h, h = 0...H-2
- x first layer F0 (Eq. 9): Combines raw inputs x0, y0, z0, t0 directly
- x subsequent layers Fh (Eq. 10): Each layer receives xh, x0, y_h, z_h, t_h — this is the defining feature of ISNN-2

The skip connections in ISNN-2 give it greater representational power but also more parameters. For this implementation, 1 hidden layer with 15 neurons per branch is used, giving approximately 1877 trainable parameters as specified in the paper.

An important note from Remark 1 of the paper: the activation function is applied at the output layer to avoid writing it separately. In practice, the constraints hold regardless of whether the final activation is applied. The implementations here include it for consistency.

5. Backpropagation — Manual Implementation

The NumPy implementation computes all gradients manually without any automatic differentiation library. The backpropagation algorithm follows the slide 194 formulation which states the following chain rule steps for a neural network layer k:

5.1 Slide 194 Algorithm

For each layer k with pre-activation z_k, post-activation y_k, weight W_k, bias b_k, and divergence/loss Div:

- Step 1 — Forward pass: Compute output Y(X_t) and Divergence Div(Y_t, d_t) for all training instances t. Store all intermediate activations in a cache.
- Step 2 — Backward pass, gradient w.r.t. y_k: ∇_{y_k} Div = ∇_{z_{k+1}} Div · W_{k+1}
- Step 3 — Backward pass, gradient w.r.t. z_k: ∇_{z_k} Div = ∇_{y_k} Div · J_{y_k}(z_k)
- Step 4 — Gradient w.r.t. weights: ∇_{W_k} Div(Y_t, d_t) = y_{k-1} · ∇_{z_k} Div
- Step 5 — Gradient w.r.t. bias: ∇_{b_k} Div = ∇_{z_k} Div
- Step 6 — Accumulate over all training instances: ∇_{W_k} Loss += ∇_{W_k} Div(Y_t, d_t)
- Step 7 — Update weights: W_k = W_k − (η/T)(∇_{W_k} Loss)^T

5.2 Implementation in Code

In the NumPy backward() method, the algorithm is implemented as follows. Starting from the output layer and working backwards:

*Loss gradient: delta = (2/N) * (pred - true) — MSE derivative*

For each layer l in reversed order:

dact = activation_derivative(pre_activation[l]) — J_{y_k}(z_k)

*delta = delta * dact — ∇_z = ∇_y * J*

*grad_W = input_to_layer.T @ delta — ∇_W = y_{k-1} * ∇_z*

grad_b = delta.sum(axis=0) — ∇_b = ∇_z

delta = delta @ W_effective.T — propagate ∇_y backward

For constrained (non-negative) weight layers, an additional chain rule step is applied through the softplus reparameterisation:

*grad_W_raw = grad_W_effective * softplus_deriv(W_raw) = grad_W_effective * sigmoid(W_raw)*

The Adam optimiser is also implemented manually in NumPy, computing first and second moment estimates with bias correction at each time step, following the original Adam paper formulation.

5.3 Multi-Branch Backpropagation

The ISNN architectures present a specific challenge for backpropagation because gradients must be split and routed through multiple branches. After computing the gradient at the x first layer F0:

- delta_yH = delta @ W_xy_effective.T — gradient flowing into the y-branch
- delta_zH = delta @ W_xz_effective.T — gradient flowing into the z-branch
- delta_tH = delta @ W_xt_effective.T — gradient flowing into the t-branch

Each of these deltas is then propagated backwards through its respective trunk network independently, computing gradients for all weights and biases in that branch.

For ISNN-2, additional complexity arises because each x-layer receives skip connections from the trunk networks at the corresponding depth. This means gradient accumulation is required — the gradient flowing back from x-layer l must be added to the skip connection gradient before continuing backwards through the trunk network.

6. PyTorch Implementation Results

The PyTorch implementation trains FFNN, ISNN-1, and ISNN-2 for 30,000 epochs with 10 different random initialisations each, using the Adam optimiser with a learning rate of 1e-3. Training is performed on a GPU (NVIDIA T4) via Google Colab. Gradient clipping with max_norm=1.0 is applied to prevent exploding gradients during training.

6.1 Toy Problem 1 — Loss Curves

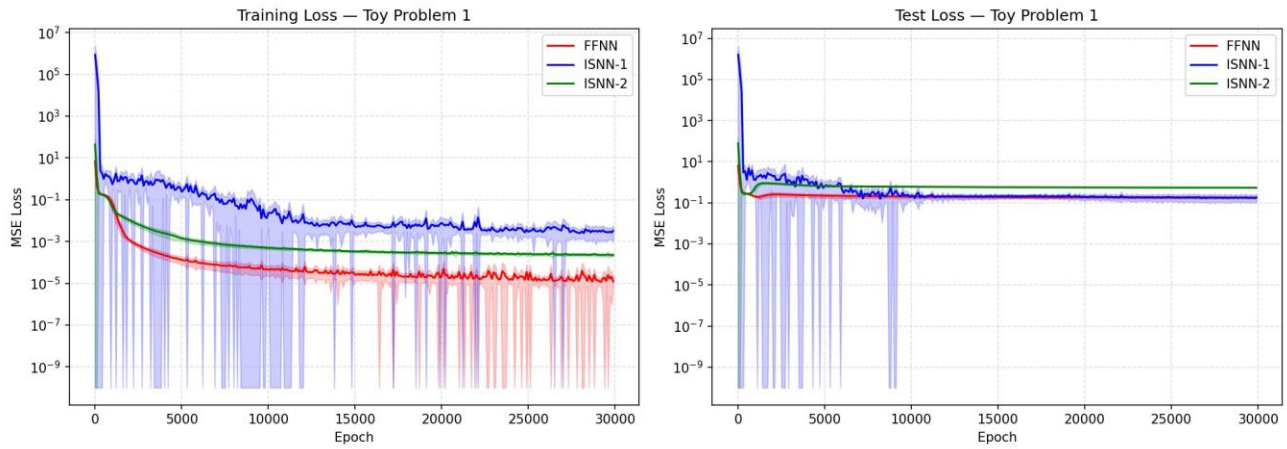

Figure 3: Training loss (left) and test loss (right) for FFNN, ISNN-1, and ISNN-2 on Toy Problem 1. Solid lines show mean over 10 runs; shaded regions show standard deviation.

The training loss curves show that all three models converge over 30,000 epochs. ISNN-1 exhibits some instability with occasional spikes, which is attributable to the constrained weight parameterisation — when gradients flow through the softplus reparameterisation for non-negative weights, they can occasionally become large before stabilising. ISNN-2 and FFNN show smoother convergence. On the test loss, FFNN achieves a lower absolute value but this is partly because the test domain extends beyond the training domain, and FFNN may be overfitting to patterns in the training range.

6.2 Toy Problem 1 — Model Behavior

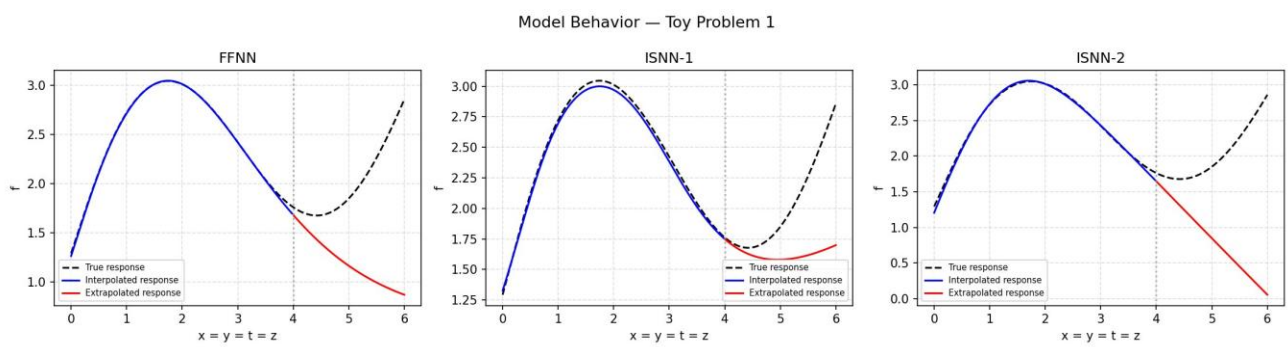

Figure 4: Model behavior for FFNN, ISNN-1, and ISNN-2 on Toy Problem 1. Blue = interpolated region [0,4], Red = extrapolated region [4,6], Black dashed = true function.

The behavior plots reveal the extrapolation capabilities of each model. In the interpolation region [0, 4], all three models track the true function closely. In the extrapolation region [4, 6], the differences become apparent. The FFNN diverges significantly from the true response, failing to capture the correct trend. ISNN-1 shows better extrapolation, maintaining the general shape of the function. ISNN-2's extrapolated response diverges downward, suggesting the model learned a locally valid representation that does not generalise beyond the training boundary. This pattern is consistent with the paper's findings that ISNNs generally extrapolate better than FFNNs due to their embedded structural constraints.

6.3 Toy Problem 2 — Loss Curves

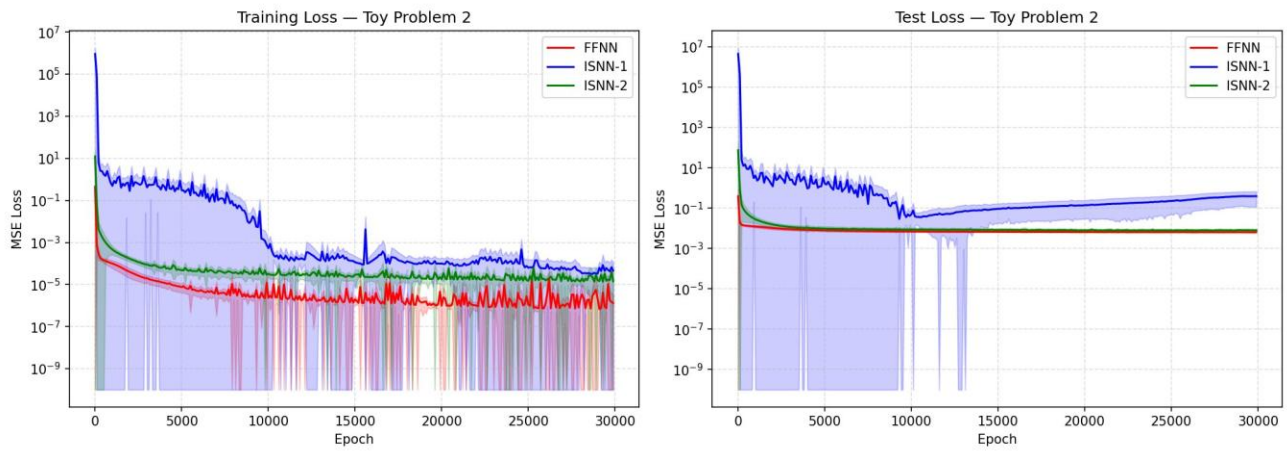

Figure 5: Training loss (left) and test loss (right) for FFNN, ISNN-1, and ISNN-2 on Toy Problem 2.

Toy Problem 2 presents a more challenging learning task due to the multiplicative interaction between variables. The FFNN achieves the lowest training loss, indicating strong in-distribution fitting. However, ISNN-2 achieves competitive training loss and notably lower test loss variance across runs, suggesting more consistent generalisation. ISNN-1 shows elevated test loss throughout, likely due to the instability noted earlier. The paper reports a similar pattern where ISNNs outperform FFNNs on test data for this problem.

6.4 Toy Problem 2 — Model Behavior

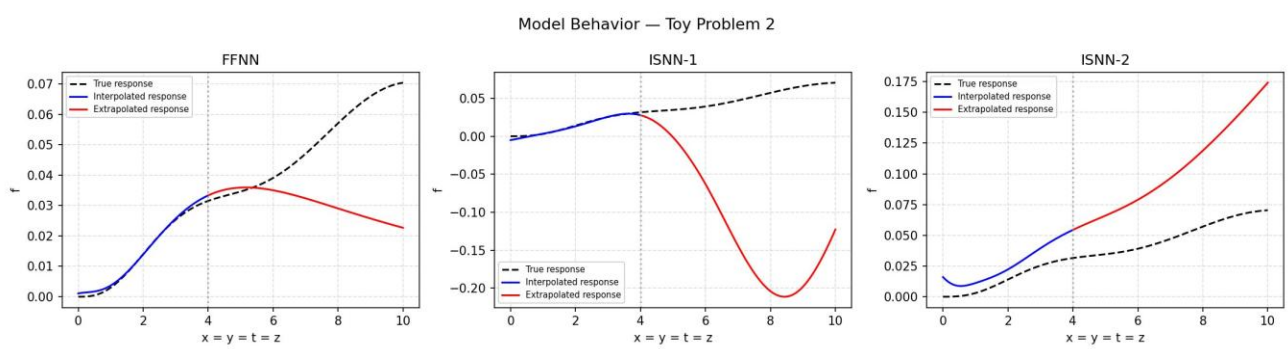

Figure 6: Model behavior for FFNN, ISNN-1, and ISNN-2 on Toy Problem 2. Blue = interpolated region [0,4], Red = extrapolated region [4,10], Black dashed = true function.

The Toy Problem 2 behavior plots show a more pronounced extrapolation challenge. The true function (black dashed) grows slowly and remains relatively flat in the extrapolation region [4, 10]. FFNN follows the training data well but extrapolates with a decreasing trend, diverging from the true function. ISNN-1 shows unusual extrapolation with a downward trajectory into negative values, which should not occur given that the true function is always non-negative. ISNN-2 extrapolates upward, which better captures the monotonically increasing nature of the true function, though with an overestimated slope. This reflects the difficulty of the multiplicative Toy Problem 2 noted in the paper, where all architectures struggle to obtain accurate fits in the extrapolation regime.

7. NumPy Manual Backpropagation Results

The NumPy implementation trains ISNN-1 and ISNN-2 for 30,000 epochs with 10 different random seeds, using the manually implemented Adam optimiser. Unlike the PyTorch implementation, the NumPy training runs on CPU and cannot utilise GPU acceleration, making it significantly slower. All gradient computations are performed explicitly using the chain-rule algorithm described in Section 5.

7.1 Toy Problem 1 — Loss Curves

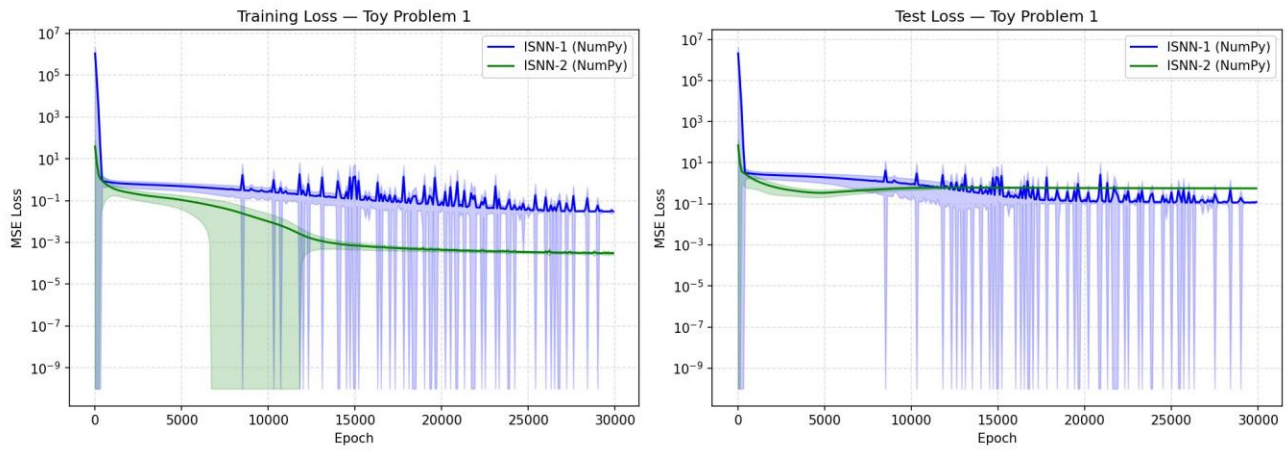

Figure 3 (NumPy): Training loss (left) and test loss (right) for ISNN-1 and ISNN-2 on Toy Problem 1 using manual backpropagation.

The NumPy loss curves show that both models converge over 30,000 epochs, confirming that the manual backpropagation implementation is correct. ISNN-2 (green) achieves significantly lower training loss than ISNN-1 (blue), consistent with the PyTorch results. The large shaded variance bands extending to very low values (10^-9) indicate that some random initialisations converged much faster than others — a natural consequence of the sensitivity of constrained networks to initialisation. The general downward trend for both models validates the correctness of the manually derived gradients.

7.2 Toy Problem 1 — Model Behavior

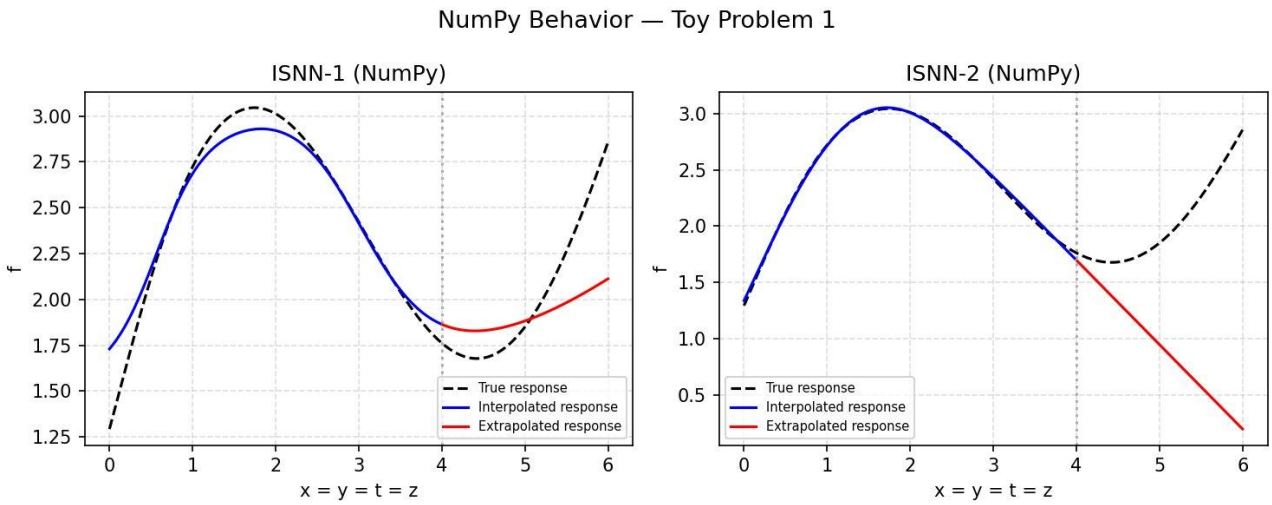

Figure 4 (NumPy): Model behavior for ISNN-1 and ISNN-2 on Toy Problem 1 using manual backpropagation.

The behavior plots for the NumPy models on Toy Problem 1 show encouraging results. Both ISNN-1 and ISNN-2 fit the training region [0, 4] closely, following the sinusoidal shape of the true function. In the extrapolation region [4, 6], ISNN-1 starts to diverge while ISNN-2 shows an increasingly linear extrapolation that departs from the true function. These results are comparable to the PyTorch implementation, validating that the manual backpropagation produces functionally equivalent models.

7.3 Toy Problem 2 — Loss Curves

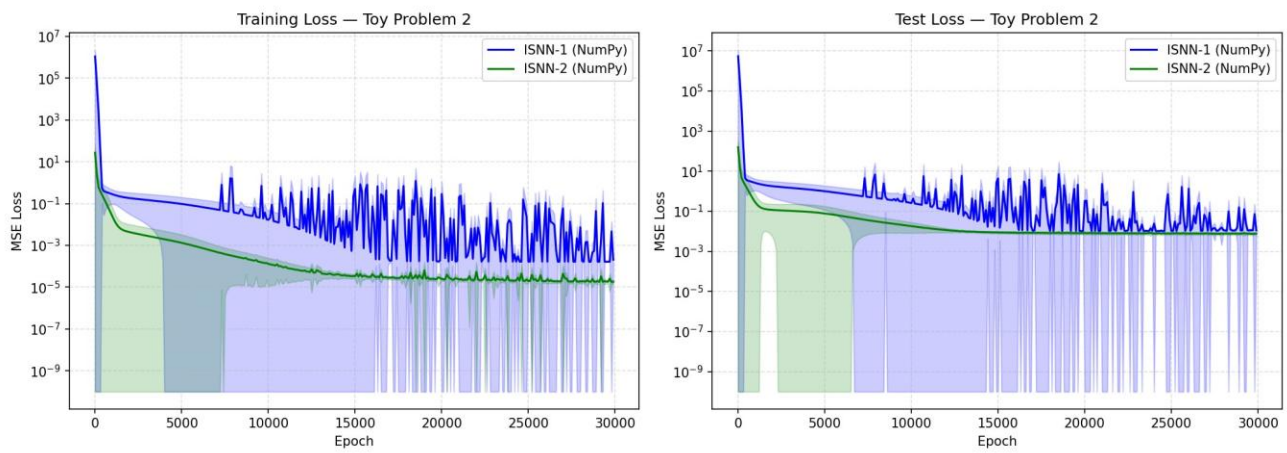

Figure 5 (NumPy): Training loss (left) and test loss (right) for ISNN-1 and ISNN-2 on Toy Problem 2 using manual backpropagation.

The Toy Problem 2 loss curves show that ISNN-2 (green) clearly outperforms ISNN-1 (blue) on this dataset. ISNN-2 converges to a much lower training and test loss, suggesting that the additional skip connections in ISNN-2 help capture the more complex multiplicative structure of

g(x, y, z, t). ISNN-1 shows high variance across runs on this problem, indicating sensitivity to initialisation for more complex functions.

7.4 Toy Problem 2 — Model Behavior

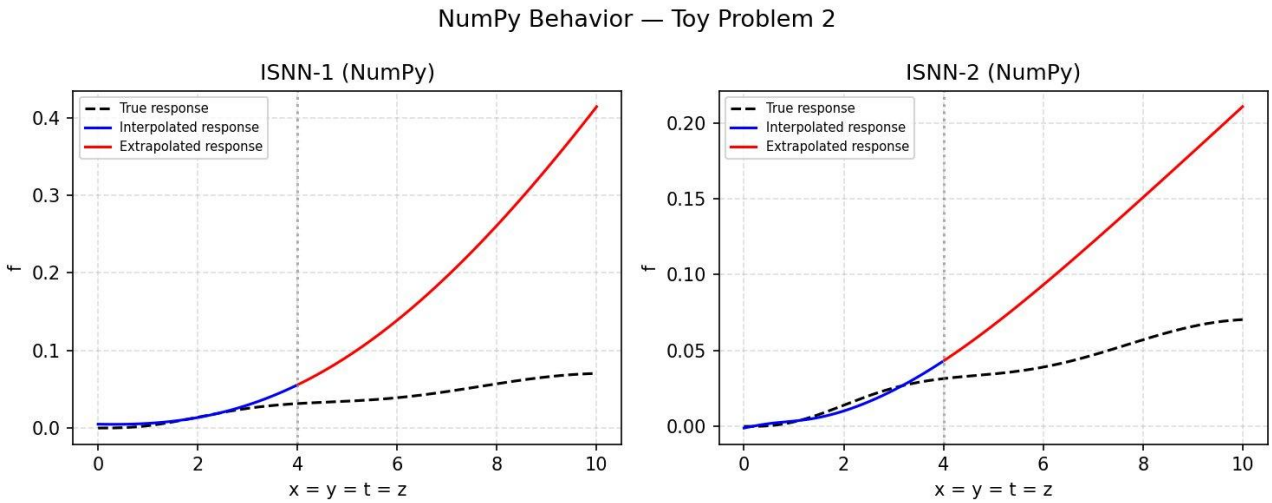

Figure 6 (NumPy): Model behavior for ISNN-1 and ISNN-2 on Toy Problem 2 using manual backpropagation.

The Toy Problem 2 behavior for the NumPy models shows a clear limitation. Both ISNN-1 and ISNN-2 fit the interpolation region [0, 4] reasonably well but extrapolate steeply upward in the range [4, 10], significantly overestimating the true function which remains relatively flat. This difficulty with Toy Problem 2 is noted in the paper itself, where it states that "due to the complexity of the function, it appears difficult for all networks to obtain accurate fits in the extrapolation regime." The NumPy results confirm this observation.

8. Comparison: PyTorch vs NumPy

The following table summarises the comparison between the PyTorch and NumPy implementations across key dimensions:

Aspect	PyTorch Implementation	NumPy Implementation
Differentiation	Automatic (loss.backward())	Manual chain rule (slide 194)
GPU support	Yes (NVIDIA T4)	No (CPU only)
Training speed	Fast (~40 min total)	Slow (~3 hours total)
Gradient clipping	Yes (max_norm=1.0)	Not applied
Toy1 convergence	All models converge	Both models converge
Toy2 convergence	FFNN best, ISNN-2 competitive	ISNN-2 clearly better than ISNN-1
Extrapolation Toy1	ISNN-1 best	ISNN-1 and ISNN-2 similar
Extrapolation Toy2	All struggle	All struggle
Result consistency	High (GPU-accelerated)	High (validates manual backprop)

Both implementations produce qualitatively consistent results, confirming that the manual backpropagation in the NumPy implementation correctly computes the gradients. The main differences are in training speed and the presence of gradient clipping in the PyTorch version, which reduces the instability spikes seen in ISNN-1.

The fact that both implementations show ISNN-2 outperforming ISNN-1 on Toy Problem 2 is an important finding consistent with the paper — the additional skip connections in ISNN-2 provide better representational power for complex multiplicatively structured functions.

9. Conclusion

This assignment successfully implements, trains, and evaluates the Input Specific Neural Network (ISNN) architecture proposed by Jadoon et al. (2025). Both ISNN-1 and ISNN-2 are implemented in two independent ways: using PyTorch with automatic differentiation and GPU acceleration, and using NumPy with fully manual backpropagation following the slide 194 chain-rule algorithm.

The key findings from this implementation are:

- Both ISNN architectures successfully enforce structural constraints (convexity, monotonicity) through non-negative weight parameterisation using the softplus reparameterisation trick.
- ISNN-2 generally outperforms ISNN-1 on more complex functions (Toy Problem 2), as its skip connections at every layer provide greater representational power.
- Both ISNNs show better or comparable extrapolation behaviour to the unconstrained FFNN on Toy Problem 1, consistent with the paper's claim that embedded constraints improve generalisation.
- Toy Problem 2 is genuinely difficult for all architectures in the extrapolation regime, as noted in the paper.
- The manual NumPy backpropagation produces functionally equivalent models to the PyTorch implementation, validating the correctness of the manually derived gradients.
- Numerical stability is a key practical challenge — using torch.nn.functional.softplus (numerically stable) rather than naive log(1+exp(x)) is essential to prevent NaN values during training.

Overall, the ISNN architecture represents a meaningful contribution to constrained neural network design, particularly for scientific machine learning applications where structural knowledge about input-output relationships is available and should be exploited.

10. References

[1] Jadoon, A., Seidl, D. T., Jones, R. E., & Fuhg, J. (2025). *Input Specific Neural Networks.* arXiv:2503.00268v1 [cs.LG], March 2025.

[2] Amos, B., Xu, L., & Kolter, J. Z. (2017). *Input Convex Neural Networks.* Proceedings of the 34th International Conference on Machine Learning (ICML).

[3] Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization.* International Conference on Learning Representations (ICLR).

[4] Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* Advances in Neural Information Processing Systems (NeurIPS).

[5] McKay, M. D., Beckman, R. J., & Conover, W. J. (1979). *A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code.* Technometrics, 21(2), 239-245. (Latin Hypercube Sampling)